

MISC-04 — Local Disaster Warning & Response Coordination Platform

Technical Workflow Specification (Professional-Grade, 80% Build)

Everything below must be computed from real, sourced data (real rainfall figures, real terrain/elevation values, real settlement coordinates, real graph structure) — not randomly assigned risk levels or hardcoded demo values. The differentiator judges are grading against is explicitly stated in the PS: *"shows the evidence and confidence behind each warning"* and *"tested on false-alert rate and route validity."* If risk scores are hand-picked per settlement instead of derived from real feature data, this collapses into a static mockup the moment a judge asks "why is this one red?" — so every number on screen must trace back to a real input.

1. Shared State Contract (`state.json`) — lock before anyone writes code)

This is the single interface all three members build against. Nobody edits another person's internals — only this shape.

```
json
```

```
{
  "district": "string",
  "hazard_type": "flood",
  "timestep_index": 0,
  "timesteps": ["T1", "T2", "T3", "T4", "T5", "T6", "T7", "T8"],
  "data_label": "REPLAYED – sourced from IMD historical rainfall, [date range]",

  "settlements": [
    {
      "id": "S01",
      "name": "string",
      "lat": 0.0, "lng": 0.0,
      "elevation_m": 0,
      "distance_to_river_km": 0.0,
      "population": 0,
      "historical_flood_flag": 0,
      "rainfall_mm": { "T1": 0, "T2": 0, "...": 0 },
      "risk_score": { "T1": 0.0, "...": 0.0 },
      "risk_level": { "T1": "low", "...": "high" },
      "confidence": { "T1": 0.0, "...": 0.0 },
      "priority_rank": { "T1": 1, "...": 5 }
    }
  ],

  "roads": [
    { "id": "R01", "from": "S01", "to": "S02",
      "coordinates": [[lat,lng],[lat,lng]],
      "status": { "T1": "open", "...": "closed" } }
  ],
```

```

"ground_reports": [
  { "id": "G01", "settlement_id": "S03", "timestep": "T3",
    "type": "flooded_road|stranded_people|other", "text": "string" }
],

"hazard_alerts": [
  { "id": "A01", "settlement_id": "S03", "timestep": "T4",
    "risk_level": "high", "confidence": 0.0,
    "evidence": { "rainfall_mm": 0, "elevation_m": 0,
                  "distance_to_river_km": 0.0, "ground_reports_nearby": 0 },
    "label": "string" }
],

"test_scenarios": {
  "false_alert_case": { "description": "string", "result": "string", "passed": true },
  "route_disruption_case": { "description": "string", "result": "string", "passed": true }
}
}

```

Every field above must be filled from a real computation, not a placeholder left in the final build. Placeholder data is fine on Day 1 for parallel development — it must not survive into the Day 2 integration.

2. Member A — Data Sourcing & Real Risk Model

2.1 Real data acquisition (not synthetic where avoidable)

- **Rainfall:** pull real historical daily/hourly rainfall for the chosen district from IMD or data.gov.in. Select one real historical high-rainfall period as the backbone of your 8 timesteps — this is what lets you honestly say "replayed real event," not "invented numbers."
- **Settlements:** hand-pick 15-20 real named villages/wards in the district using OpenStreetMap or Google Maps — real names, real lat/lng, real approximate population (Census 2011 village-level data is public and searchable per district).

- **Elevation:** query Open-Elevation API (free, no key) per settlement coordinate — real values, not estimated.
- **Distance to river:** compute real geodesic distance from each settlement's coordinates to the nearest river polyline (can trace the river manually from OSM and compute point-to-line distance with `shapely`).
- **Historical flood flag:** binary flag per settlement — set to 1 only if you find real evidence (news reports, district disaster management records) that the settlement has flooded before; otherwise 0. Document your source per settlement in a comment/footnote so this isn't fabricated.

2.2 Timestep construction (real event, replayed honestly)

Take your real rainfall time series for the chosen historical event and bucket it into 8 timesteps (T1 = early/light rain building to T8 = peak). Each settlement's `rainfall_mm[Ti]` should be the real recorded value for that time window at the nearest rain-gauge/grid-cell to that settlement — not the same number applied to every settlement.

2.3 Risk score model (the actual AI/ML component)

Train a real model — logistic regression or random forest via scikit-learn — using these features:

```
X = [rainfall_mm, elevation_m, distance_to_river_km, historical_flood_flag]
y = flood_occurred (0/1) # from historical event labels, or a rule-derived proxy if no labeled ground truth exists – document which
```

If you cannot find real labeled flood-occurrence ground truth for training, construct an honest **proxy label** using a documented physical rule (e.g. `y=1 if rainfall_mm > threshold AND elevation_m < threshold AND distance_to_river_km < threshold`) and say explicitly in your PPT/README that the model is trained on a physically-motivated proxy label, not verified ground-truth flood records. This is more defensible under judge questioning than silently presenting it as verified.

Output per settlement per timestep:

- `risk_score` = model's predicted probability (0.0-1.0)
- `risk_level` = `low` (<0.33) / `medium` (0.33-0.66) / `high` (>0.66)

- `confidence` = model's predicted-probability distance from 0.5, rescaled to 0-1 (`confidence = abs(risk_score - 0.5) * 2`) — a real, derivable uncertainty measure: predictions near 0.5 are genuinely less certain, predictions near 0 or 1 are genuinely more certain. Document this formula in a code comment.

2.4 Validation (required, not optional)

Run and document two real test cases against your trained model:

- **False-alert case:** a settlement/timestep with high rainfall but high elevation + far from river — confirm the model does NOT flag it high-risk. Record the actual output.
- **Disruption case:** a settlement/timestep combination that should knock out a road connection — confirm downstream road-status logic (Member B) correctly reflects it once your `risk_score` is fed in.

Deliverable: `data/` folder with sourced CSVs (rainfall, settlements, elevation), `model.py` (training + inference function), `state_settlements.json` (all real computed fields filled), and a `MODEL_README.md` documenting feature choices, the proxy-label decision if used, exact confidence formula, and the two test results with numbers.

3. Member B — Coordination Logic, Ranking & Safe Routing

3.1 Road graph (real structure, not visual-only lines)

Build the settlement/road network as an actual `networkx.Graph()` — nodes = settlements, edges = roads with real (or realistically traced from OSM) coordinates. This is not decorative: the graph must be queryable for shortest-path and connectivity, which the safe-route feature and disruption test both depend on.

3.2 Road status derivation (rule, real inputs)

```
road.status[Ti] = "closed" if (risk_level[from][Ti] == "high" OR risk_level[to][Ti] == "high")
                        OR (a ground_report of type "flooded_road" exists near this road at Ti)
                        else "open"
```

Document this exact rule — it must be traceable, not opaque, per the PS requirement that every alert carry evidence.

3.3 Priority ranking formula (real, weighted, documented)

```
priority_score = (0.5 * risk_score) + (0.3 * normalized_population) + (0.2 * (1 - road_accessibility_score))
```

Where `road_accessibility_score` = fraction of a settlement's connected roads currently open (0 = fully cut off, 1 = fully connected). Rank settlements descending by `priority_score` per timestep → this becomes `priority_rank`. State the weights explicitly in your README; judges specifically reward showing the reasoning behind a ranking rather than an unexplained sort.

3.4 Ground-report influence (real signal integration, not decorative)

When a ground report lands near a settlement at timestep T_i , bump that settlement's effective `risk_score` by a small, documented delta (e.g. +0.1, capped at 1.0) before ranking — this demonstrates the PS's requirement to combine "environmental observations and reports from people on the ground," not just model output alone.

3.5 Safe-route feature (bonus, cheap given the graph already exists)

```
python
nx.shortest_path(graph, source=affected_settlement, target=nearest_low_risk_settlement,
                 weight=lambda u,v,d: float('inf') if d['status']=='closed' else 1)
```

Return the path as a coordinate list for Member C to draw. This directly reuses the Section 3.1 graph — no new data needed.

3.6 Alert generation (evidence-carrying, per PS requirement)

For every settlement crossing `risk_level == high` at a given timestep, push one entry into `hazard_alerts` with the real `evidence` block (actual `rainfall_mm`, `elevation_m`, `distance_to_river_km`, count of nearby ground_reports at that timestep) and a generated `label` string, e.g. `"High flood risk – 3 nearby reports, road R04 closed"`. Never emit an alert with an empty evidence block.

Deliverable: `backend/graph.py`, `backend/ranking.py`, `backend/routing.py`, `backend/alerts.py`, plus the fully populated `roads` and `hazard_alerts` sections of `state.json`, and a short README stating the ranking formula weights and the road-closure rule.

4. Member C — Viewer, Interactivity & Evidence Presentation

4.1 Map rendering (Leaflet)

- Settlement markers colored by `risk_level[current_timestep]` — real per-point recoloring driven by `state.json`, not a fixed palette assigned once.
- Road lines colored by `status[current_timestep]` — green (open) / red (closed).
- A persistent, unmissable banner: `"REPLAYED DATA – [real event date range]"`, sourced from `data_label` in the state contract. This must never be silently dropped, since it's an explicit acceptance criterion in the PS (never confused with live data).

4.2 Timestep control (the "conditions change" requirement)

A slider or play button stepping through `T1`→`T8`. On each step: re-color markers, re-color roads, re-sort the alert sidebar by `priority_rank` — all read live from `state.json`, never hardcoded per-step visuals. This is what proves the platform "updates response priorities as conditions change," not a static screenshot with a slider bolted on for show.

4.3 Alert sidebar (evidence-first, per PS requirement)

Each alert row must display the real `evidence` block inline — rainfall, elevation, distance to river, nearby-report count — and the `confidence` score, not just a risk-level badge. Clicking an alert pans/zooms the map to that settlement.

4.4 Safe-route overlay (bonus)

A toggle that, given a selected affected settlement, draws Member B's computed shortest-path line to the nearest low-risk settlement on the map, using only currently-open roads.

4.5 Legend & live summary panel

Fixed panel showing: color-key for risk levels and road status, plus a running computed line: `"X settlements high-risk · Y roads closed · Z ground reports active"` — recalculated live from the real current-timestep state, not a static string.

4.6 Test-scenario display

A small panel or PPT slide (whichever is faster) showing the two real test results from Member A/B's validation (Section 2.4): the false-alert case and the disruption case, with actual numbers, not a claimed-but-unshown "we tested this."

Deliverable: `frontend/` map app wired to `state.json`, PPT (official template) with architecture diagram + test-result screenshots, 3-4 min video script and recording, GitHub README tying all three members' modules together, and the `TeamName_PSno` Drive folder.

5. Honesty Notes (keep these explicit in the PPT/README — judges reward this)

- If real labeled flood-occurrence data wasn't available for training, say so and name the proxy-label rule used instead — this is more credible than an unstated model.
 - Don't claim live/real-time capability — the system uses **real historical data, replayed and clearly timestamped**, which the PS explicitly permits and rewards when labeled correctly.
 - Don't claim the model "predicts floods" in general — frame it as "estimates relative flood risk for this district based on the sourced historical event," since that's what the data and validation actually support.
-

6. Acceptance Criteria (must hold true when demoed)

- Changing the input rainfall data for a settlement must visibly change its `risk_score`, `risk_level`, and its position in the priority ranking — nothing is hardcoded per settlement.
- The false-alert test case must show the model correctly withholding a high-risk label despite high rainfall, when elevation/river-distance say otherwise.
- Advancing the timestep slider must visibly re-color at least one road from open to closed and re-order the alert list — proving the "conditions change" requirement is real, not decorative.
- The safe-route feature must only ever route through roads marked `open` at the currently selected timestep.
- Every alert on screen must carry a non-empty, real evidence block.